Introduction to Programming in R (Two-Week Module)

Follows: Introduction to R & RStudio/R Markdown

Data Science for the Psychological Sciences

January 05, 2026

Contents

Module overview

Learning outcomes

1. Write, test, and reuse simple **functions** (including argument defaults).
2. Use **control flow** (`if`, `else if`, `else`) to implement decision logic.
3. Use **iteration** (`for`, `while`) appropriately and explain when vectorization is preferable.
4. Import a dataset, clean variable names, handle missing values, and compute group summaries.
5. Create publication-ready plots using **ggplot2**.
6. Write a short, reproducible analysis report in **R Markdown** with narrative + code + outputs.

A psychology-style dataset for consistent practice

This module uses a small synthetic (simulated) dataset that mimics a common psychology experiment with reaction times and accuracy. Each row represents one participant (id), who is assigned to either a control or treatment condition. The dataset includes a continuous outcome (`rt_ms`, reaction time in milliseconds), a binary outcome (`accuracy`, correct/incorrect), and two common individual-difference variables (`stress` on a 1–7 Likert-style scale and `sleep_hours`). We generate the values using probability models (normal distributions for continuous measures and a binomial model for accuracy), and we apply simple constraints to keep them realistic (e.g., RTs cannot be implausibly fast, and stress stays within 1–7). Creating a dataset like this is important because it gives everyone a consistent, safe, and fully reproducible set of data to practice on without needing to locate files, handle privacy restrictions, or worry that their results differ because their data differ.

The code used to generate the dataset and the first few rows of the dataset are illustrated below:

```
set.seed(123)

n <- 120
psych_data <- tibble(
  id = 1:n,
  condition = sample(c("control", "treatment"), n, replace = TRUE),
  rt_ms = round(rnorm(n, mean = 650, sd = 120)),
  accuracy = rbinom(n, size = 1, prob = 0.82),
  stress = round(rnorm(n, mean = 4.2, sd = 1.6), 1),# Likert-ish scale (1-7)
  sleep_hours = round(rnorm(n, mean = 6.8, sd = 1.1), 1)
) %>%
  mutate(
    rt_ms = pmax(rt_ms, 250),
    stress = pmin(pmax(stress, 1), 7)
  )

print(head(as.data.frame(psych_data)))
```

```
##   id condition rt_ms accuracy stress sleep_hours
## 1  1   control   696        1    2.9         6.6
## 2  2   control   590        1    3.4         7.5
## 3  3   control   610        0    6.6         7.1
## 4  4 treatment   528        1    2.4         7.9
## 5  5   control   521        1    3.9         7.7
## 6  6 treatment   686        1    7.0         6.6
```

1 Scripts vs console + the first functions

An R script is a saved, reusable record of your analysis steps, written in an R script file (e.g., .R) or an R Markdown document, so you can run the same code again later, review what you did, and/or share it with someone else for replication. Using the command line (the Console) is more like "working live": you type commands interactively and see results immediately, which is useful for quick checks, exploring objects, or testing a small idea. The tradeoff is that console work is easy to lose, hard to audit, and difficult to reproduce exactly unless you copy the final code into a script. In practice, the recommended workflow is: use the Console for small experiments, but put anything that matters for your analysis (data cleaning, transformations, plots, statistics) into a script so the full workflow can be rerun end-to-end.

Like a script, an R **function** is a reusable chunk of code, but one that (1) takes one or more inputs (called arguments), (2) performs a set of operations with the input(s), and (3) returns an output. Functions are how you avoid copy-and-paste programming: instead of rewriting the same steps every time (e.g., cleaning a variable, computing a score, producing a standard summary table), you write the logic once, give it a clear name, and call it whenever you need it. This makes your work more consistent and less error-prone, because changes only need to be made in one place. Functions also make your analysis easier to read: a well-named function (e.g., `z_score(rt_ms)` or `summarize_by_condition(data)`) communicates intent immediately, while the details stay inside the function body. Over time, building small, focused functions becomes one of the main ways you move from "running commands" to writing clean, reproducible analysis code.

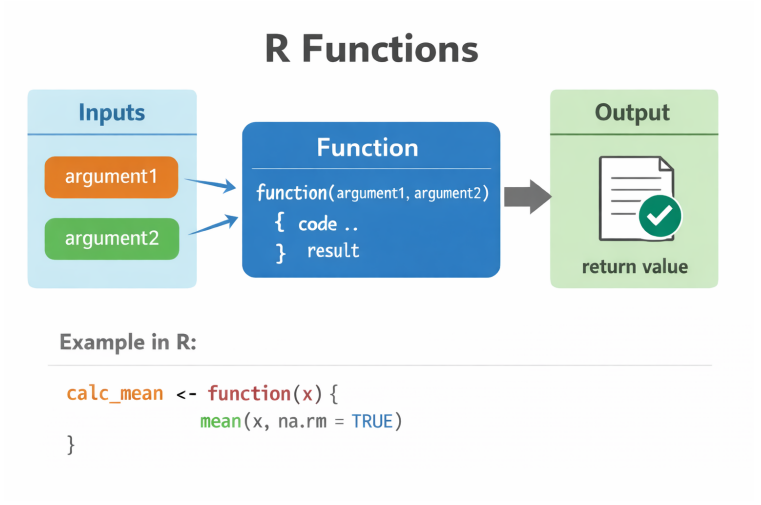

Figure 1: Anatomy of functions in R.

When you call a function in R, its arguments (the names inside the parentheses) behave like temporary variables that exist only while the function is running. Think of a function as a small "mini-workspace": when you run `my_fun(x = 10)`, R creates a local copy of x inside the function and uses it to do the work. That `x` is not the same as any `x` you might have in your Global Environment, and changing x inside the function does not automatically change your outside `x`. When the function finishes, those temporary argument variables (and anything else created inside the function) are discarded, and the only thing that comes back out is what the function returns. This is a major reason functions are so useful: they keep code organized and prevent intermediate variables from cluttering your workspace or accidentally overwriting objects you care about.

In order fo futher illustrate the difference between a script and a function, lets consider how each could be used to convert degrees in Celsius to degrees in Fahrenheit at the beginning and at the end of an experiment.

A script to convert from Celsius to in Fahrenheit is illustrated below:

```
DegreesCelcius=0
DegreesFahrenheit=(DegreesCelcius * 9/5) + 32
```

This script is a simple code chunk that you could save for future use or to share with others. Note, however, that this script *only* computes the conversion for a single value of the `DegreesCelcius` variable. Sure, you could change the value of `DegreesCelcius` and re-run the code chunk, but this would overwrite your original calculations, potentially making them impossible to replicate later.

Another possibility is that you could copy-paste the script contents in order to keep a more complete record of your calculations (see below).

```
DegreesCelciusBeginning=0
DegreesFahrenheitBeginning=(DegreesCelciusBeginning * 9/5) + 32

DegreesCelciusEnd=10
DegreesFahrenheitEnd=(DegreesCelciusEnd * 9/5) + 32
```

This solution can work, but is prone to errors and very inefficient. For example, each time the two-line computations are copied, there are three different places where variable names must be changed (e.g., `DegreesCelciusBeginning` and `DegreesCelciusEnd`) in order for the script to work correctly. Another potential issue that might occur is that, in the event you discover that your calculations included even a minor error, corrections to those calculations must be implemented at every instance of the copied script.

A much simpler, more efficient, and less error prone approach is to use a function. Consider the example below:

```
# A simple converter function
celsius_to_fahrenheit <- function(celsius) {
  (celsius * 9/5) + 32
}

DegreesCelciusBeginning <- celsius_to_fahrenheit(0)
DegreesCelciusEnd <- celsius_to_fahrenheit(10)
```

A function solves the above-mentioned problems by letting you write the conversion once and then reuse it as many times as you like with different inputs. Here, the function celsius_to_fahrenheit() takes a single argument, `celsius`, which acts like a temporary variable inside the function: when you call `celsius_to_fahrenheit(0)`, the value 0 is used as celsius for the duration of that calculation, and then the function returns the Fahrenheit result. This means you no longer need to create multiple versions of the same variable name or duplicate the same lines of code. If you later realize the formula needs to change (for example, to handle rounding etc.), you need only update the function in one place and every call automatically uses the corrected logic. In other words, functions reduce repetition, prevent copy-paste mistakes, and make your work easier to read, reuse, and replicate.

2 Functions with multiple inputs and outputs

Many useful functions take more than one input (multiple arguments). This allows you to write one reusable "recipe" that can be adapted to different situations. For example, a function that clips values to a plausible range needs three inputs: the data (x) and the lower and upper bounds (low, high). In R, you can designate arguments by position (in the order arguments appear) or by name (using arg = value).

Named arguments are especially helpful once functions have several inputs, because they make your code more readable and reduce the chance of mixing up values. An example of a function taking multiple input arguments is illustrated below.

```
clip_to_range <- function(x, low, high) {
  x[x < low] <- low
  x[x > high] <- high
  x
}

clip_to_range(c(-2, 1, 3, 10), 0, 7)                  # by position
clip_to_range(c(-2, 1, 3, 10), low = 0, high = 7)     # by name
```

2.1 Default arguments and "optional" inputs

Often, one or more inputs have a typical value. In that case, you can set a default argument so the function works even if the user does not supply that value. Defaults make functions easier to use and help you avoid repeating the same numbers throughout your script. For example:

```
clip_to_range <- function(x, low = 1, high = 7) {
  x[x < low] <- low
  x[x > high] <- high
  x
}

clip_to_range(c(0, 2, 9))                  # uses defaults low=1, high=7
clip_to_range(c(0, 2, 9), low = 0, high = 10)  # overrides defaults
```

A practical rule: when calling a function with multiple inputs, use named arguments whenever it improves clarity.

3 "Multiple outputs" in R: returning lists (or data frames)

In R, a function technically returns one object, but that object can contain multiple pieces of information. The most common way to return multiple outputs is to return a list (or a tibble/data frame). This is extremely useful in data analysis because you often want both a result and supporting information (e.g., a mean and an SD, or a model and its predictions).

Below is an example of a function that returns several descriptive statistics at once:

```
describe_rt <- function(rt_ms) {
  list(
    n = sum(!is.na(rt_ms)),
    mean = mean(rt_ms, na.rm = TRUE),
    sd = sd(rt_ms, na.rm = TRUE),
    min = min(rt_ms, na.rm = TRUE),
    max = max(rt_ms, na.rm = TRUE)
  )
}

rt_summary <- describe_rt(psych_data$rt_ms)
rt_summary
```

```
rt_summary$mean
rt_summary$sd
```

Notice how we access list elements using $ (e.g., `rt_summary$mean`). This keeps related outputs bundled together and prevents your workspace from filling up with many separate variables.

3.1 Functions Practice

1. Write a function `minutes_to_seconds(mins)`.

- Test it on 1, 2.5, and 10.

2. Write a function `average_of_numbers(numbers)`.

- Test it on `c(2,9,4,6)`.
- Compare the output of your function with the `mean()` function.

3. Write a function `descriptive_stats(numbers)` that returns the minimum, maximum, and range of a vector of numeric values.

- Test it on `c(2 34 2 9 15 21)`

4. Write a function 'proportion_above(x, cutoff)' that returns the proportion of values in vector x that are strictly greater than cutoff.

- Test: proportion_above(c(1,2,3,4,5), cutoff = 3),

---

4 Debugging and "reading your own code"

As you begin to write longer scripts and functions, "debugging" will become an increasingly important skill. Debugging is the process of figuring out why code is not doing what you expect and then fixing it in a way that prevents the same issue from happening again. For new programmers, the most important skill is not "never making mistakes," but learning to read your own code carefully and test your assumptions step by step. A good debugging mindset is to treat your script like a set of claims: "this object exists," "it has the type I think it has," "it contains the values I think it contains," and "this function is receiving the inputs I intended." When something breaks, resist the urge to change multiple things at once. Instead, identify the first line where the output becomes surprising, and then work forward in small increments until you find the source of the problem.

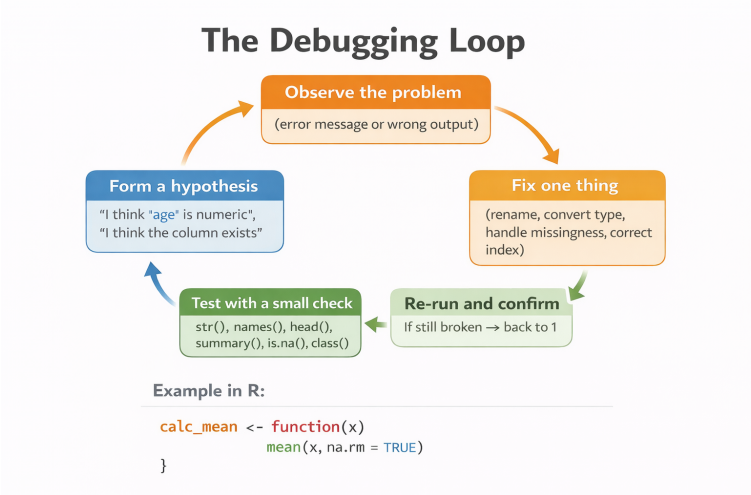

Figure 2: The process of debugging code.

In practice, this means using simple diagnostic tools frequently. `print()` (or just typing an object name in the Console) is useful for quickly checking intermediate results—especially inside longer pipelines or functions—so you can confirm what a variable contains at that moment. `str()` tells you the structure of an object (data type, columns, and classes), which is often the fastest way to spot issues like a numeric variable that was accidentally imported as character. `summary()` gives you a compact overview of ranges, missingness, and typical values, which helps catch problems like impossible scores or unexpectedly many NAs. `View()` is helpful for visually scanning a data frame to spot patterns, but it should be used as a supplement to `str()` and `summary()`, not a replacement. Finally, when you see an error message, focus on the specific complaint (e.g., "object not found," "non-numeric argument," "subscript out of bounds") and identify which line triggered it; the error is usually telling you exactly which assumption failed.

---

5 Flow Control and Loops

Flow control is what makes a script behave less like a fixed checklist and more like a set of rules that can adapt to different situations. Up to this point, most of the code you have written runs from top to bottom in the same way every time. That works when every dataset is perfectly formatted and every task is identical, but real research data rarely this way. Sometimes a variable contains missing values, sometimes a column name is slightly different than expected, and sometimes you need to apply the same operation to many participants, conditions, or files. Flow control gives you the tools to handle those situations in a systematic way rather than by manual editing and repeated copy-paste.

In R, flow control mainly comes in two forms. Conditionals (if, else if, else) let your code make decisions: if a value is missing, do one thing; if it is outside a plausible range, do another; otherwise proceed normally. Loops (for, while) let your code repeat a set of steps automatically: compute the same statistic for each condition, run the same analysis across multiple variables, or simulate an experiment hundreds of times to understand variability. Used well, these tools make your work faster, more consistent, and easier to reproduce because the "decision rules" and "repetition rules" live in your script, not in a series of manual steps you performed once and then forgot.

5.1 Conditional Statements (`if`, `else if`, `else`)

Conditional statements let you run specific chunks of code only if certain conditions are met.

5.1.1 The `if` Statement

The simplest conditional is an `if` statement. The code inside the curly braces `{}` will only run if the condition in the parentheses `()` evaluates TRUE.

```
x <- 15

if (x > 10) {
  print("The variable x is greater than 10.")
}
```

```
## [1] "The variable x is greater than 10."
```

Since `x` is 15 (which is greater than 10), the message was printed. If `x` was 5, nothing would have happened.

5.1.2 Adding `else`

You can add an `else` statement to provide an alternative block of code to run in the event that the `if` condition evaluates FALSE.

```
temperature <- 12 # in Celsius

if (temperature > 25) {
  print("It's a hot day!")
} else {
  print("It's not a hot day.")
}
```

```
## [1] "It's not a hot day."
```

5.1.3 Chaining Conditions with `else if`

You can check multiple conditions in a sequence using `else if`. R will check them one by one and run the code for the *first* one that is TRUE. If none are true, the final `else` block is run.

```
grade <- 85

if (grade >= 90) {
  print("You got an A!")
} else if (grade >= 80) {
  print("You got a B.")
} else if (grade >= 70) {
  print("You got a C.")
} else {
  print("You need to study more.")
}
```

```
## [1] "You got a B."
```

5.2 Loops (`for` and `while`)

Loops are used to repeat a block of code multiple times without having to write it out over and over.

5.2.1 The `for` Loop

A `for` loop is perfect for when you want to iterate over every item in a sequence (like a vector).

Example 1: Looping over numbers This loop will go through the `numbers` vector, and in each iteration, the current item will be assigned to the variable `num`.

```
numbers <- c(1, 2, 3, 4, 5)

for (num in numbers) {
  squared_num <- num^2
  print(paste("The square of", num, "is", squared_num))
}
```

```
## [1] "The square of 1 is 1"
## [1] "The square of 2 is 4"
## [1] "The square of 3 is 9"
```

```
## [1] "The square of 4 is 16"
## [1] "The square of 5 is 25"
```

Example 2: Looping over text The same logic applies to character vectors.

```
fruits <- c("apple", "banana", "cherry")

for (fruit in fruits) {
  print(paste("I would like to eat a", fruit))
}
```

```
## [1] "I would like to eat a apple"
## [1] "I would like to eat a banana"
## [1] "I would like to eat a cherry"
```

5.2.2 The `while` Loop

A `while` loop will continue to run as long as its condition is TRUE. You have to make sure that something inside the loop changes so that the condition eventually becomes FALSE, otherwise you'll create an infinite loop!

This `while` loop implements a simple countdown.

```
countdown <- 5

while (countdown > 0) {
  print(countdown)
  # IMPORTANT: We decrease the value of countdown in each iteration.
  # This ensures the loop will eventually stop.
  countdown <- countdown - 1
}
```

```
## [1] 5
## [1] 4
## [1] 3
## [1] 2
## [1] 1
```

```
print("Blast off!")
```

```
## [1] "Blast off!"
```

This document has covered some of the fundamental building blocks to get you started on your journey with R!